

BURT GARDNER

Software Engineer | Full-Stack .NET & Azure | AI-Assisted Delivery — the model gets a voice, never a vote

Bella Vista, AR | bgard68@gmail.com | linkedin.com/in/burt-gardner | github.com/bgard68

PROFESSIONAL SUMMARY

Full-stack .NET and Azure engineer with 18 years at Walmart Global Tech who still writes and ships production code daily on a current stack — .NET 10, React and Angular, Azure, GitHub Actions. Builds LLM-assisted systems the way security tooling has to be built: the model gets a voice, never a vote. In the flagship project that constraint is enforced rather than asserted — deterministic rules are the only source of truth, and a recorded reply that obeys an injected instruction is rejected anyway by a mutation-tested containment gate. Track record of driving architecture decisions, mentoring engineers, and delivering correctness-critical systems.

TECHNICAL SKILLS

Languages: C# (14), VB.NET, .NET (8/10), Java, TypeScript, JavaScript, SQL, PowerShell, Bash

AI-Assisted Delivery: Claude Code (daily driver), GitHub Copilot, agentic coding workflows, schema-constrained LLM integration (OpenAI API), containment gates over model output, prompt-injection test corpora, mutation-tested guardrails, AI-generated code review and auditing

Backend & Architecture: ASP.NET Core (Minimal APIs), Clean/Onion/Hexagonal Architecture, CQRS (MediatR), SOLID, REST API design, Dependency Injection, JWT/OIDC auth, TOTP 2FA, webhooks (signature-verified, idempotent), async I/O, financial/decimal-precision correctness

Design Patterns: Strategy, Adapter, Abstract Factory, Repository, Mediator, Decorator (pipeline behaviors), Null Object, Composition Root, Ports & Adapters, rules-as-data / table-driven policy

Frontend: React 19, Angular 20, Blazor Server, TypeScript, Vite, Node.js

Data: MS SQL Server, Azure SQL, PostgreSQL 16, Teradata, Cosmos DB, EF Core, Dapper

Cloud & DevOps: GitHub Actions CI/CD (path-selective pipelines, SHA-pinned actions, least-privilege), OIDC deployments, Docker & Docker Compose, Azure (App Service, Static Web Apps, Key Vault, Managed Identity), Neon

Security & Quality: CodeQL, Gitleaks, Dependabot, secret scanning with push protection, dependency review, xUnit, WebApplicationFactory, Vitest, React Testing Library, Playwright, axe-core (WCAG 2.1 AA), unit / integration / end-to-end testing, OpenAPI

PROFESSIONAL EXPERIENCE

Independent Engineering Practice — Bella Vista, AR

Aug 2025 – Present

Self-directed R&D on AI-assisted software delivery — six shipped, CI-gated, publicly deployed open-source systems, including a GitHub Actions supply-chain analyzer with a mutation-tested containment gate.

Walmart Global Tech — Bentonville, AR

Jun 2007 – Aug 2025

Software Engineer II

- Delivered and supported production code daily on Walmart's high-availability Retail Link content-management platform serving global internal teams and external suppliers across a large multi-team, multi-service environment.
- Drove technical decisions with a high sense of ownership, championing adoption of Onion, Clean, and Hexagonal architecture to improve maintainability and scalability across services.
- Reduced system downtime approximately 90% through proactive monitoring and improved incident-response processes; participated directly in deployments, incident response, and root-cause analysis.
- Improved database performance approximately 40% via SQL query optimization and tuning, applying strong fundamentals in data access and correctness.
- Authored business-critical ad-hoc Teradata SQL for Legal and Compliance, delivering accurate datasets where data precision and correctness were non-negotiable.
- Partnered across product, design, and QA in cross-discipline teams to ship features and resolve production issues with minimal business disruption.
- Onboarded and mentored newer associates and participated in code reviews to enforce best practices and software craftsmanship; embedded secure SDLC practices to remediate vulnerabilities and authored technical documentation and training for operational continuity.

SELECTED PROJECTS

All CI-gated and built with AI agents under one constraint — generated code ships only after deterministic checks say it may. .NET 10 / C# with CodeQL, Gitleaks and dependency review on Azure; the data pipeline is dependency-free Python on GitHub Pages.

DevSecOps Sentinel — GitHub Actions Supply-Chain Security Analyzer

.NET 10 · React 19 · TypeScript 5.9 · read-only GitHub App · OpenAI API · MIT

github.com/bgard68/DevSecOpsSentinel · [Live demo](#)

- Built an analyzer where eleven deterministic rules are the only source of truth for GitHub Actions workflow risk — unpinned actions, excessive permissions, script injection, untrusted checkout, secret forwarding, artifact poisoning — while an optional OpenAI layer only explains findings it cannot originate.
- Enforced that constraint rather than asserting it: schema-constrained replies whose rule IDs must match the deterministic set exactly, proved against a prompt-injection fixture whose recorded reply fully obeys the attack and is rejected anyway — a rule ID the scanner never produced cannot survive containment. Mutation-tested: weakened to a count comparison, 5 of 14 containment tests fail.
- Measured at field scale across 564 workflows in 14 major OSS repositories (dotnet/runtime, pytorch, grafana, react, node): 564 of 564 parsed, 2,563 findings, and zero false positives on the clean baseline.
- Guarded by 322 .NET tests, 28 frontend tests and a 25-check HTTP smoke suite with path-selective CI; the project passes its own analyzer.

WidgetWorks — Production-Posture E-Commerce Platform

.NET 10 · Dapper · PostgreSQL 16 · React · TypeScript · Docker Compose · Azure · MIT

github.com/bgard68/WidgetWorks · [Live demo](#)

- Built a storefront to a production security posture: stock that cannot be oversold, totals the browser never gets to decide, and payments that settle out of band.
- Made the database arbitrate inventory — atomic stock reservation by conditional update inside a transaction — choosing PostgreSQL 16 over SQLite and SQL Server for those concurrency primitives.
- Implemented the full auth surface: JWT with single-use rotating refresh tokens, a per-user security stamp for revocation, TOTP 2FA, and Google sign-in.
- Put payments behind one IPaymentGateway seam (mock and Stripe test mode, synchronous and asynchronous), with buy-now-pay-later settling via signature-verified idempotent webhooks long after the request ends.
- Ran the stack on Azure free tiers behind 463 tests and an engineering handbook whose bug chapter documents thirty-nine real defects.

ADDITIONAL PROJECTS

BI-Simulator — Agentic Data Integration: 18 Sources, One Model

Python (stdlib only) · SQLite · CSV · JSON · JSONL · XML · GitHub Actions · GitHub Pages · MIT

github.com/bgard68/BI-Simulator · [Live demo](#)

- Directed an AI agent to build an 18-source unification pipeline — mixed formats conformed, joined at the order-line grain, flattened into a 45-column model behind a live cross-filtering dashboard — then put AI inside it: an LLM maps an unseen 19th source from a closed transform vocabulary, landing only when eleven deterministic gates pass — gates negative-case tested, prompt-injection canary neutralized, decision replayed by CI on every push.

Chess — Fully Client-Side Engine, Clocks & Game Library

React · TypeScript · Stockfish 18 · SQLite (OPFS) · no backend · MIT

github.com/bgard68/ClaudeChessApp · [Live demo](#)

- Shipped a chess app with no server at all — rules, Stockfish 18, clocks and a SQLite library all running client-side in workers — with 2,987 World Championship games validated by replaying all 256,826 half-moves; 494 tests plus four real-browser gates.

Net10Sudoku — Technique-Graded Generator & Hint Engine

.NET 10 · Blazor Server · Clean Architecture · Azure · MIT

github.com/bgard68/Net10Sudoku · [Live demo](#)

- Graded difficulty by the solving techniques a puzzle actually demands rather than clue count, verifying every clue removal keeps exactly one solution; deployed via GitHub Actions with OIDC and zero stored secrets, guarded by 87 tests.

Powerball & Mega Millions Ticket Auditor — Rules-as-Data Engine

.NET 10 · Angular 20 · Dapper · SQLite · DbUp · Azure · MIT

github.com/bgard68/LotteryApp · [Live demo](#)

- Modeled rule changes as data — seven Powerball eras back to 1992 — validating all 4,493 historical drawings against the era table so an undocumented rule change fails CI; chose Dapper over EF Core for a read-heavy schema, 72 tests.

EDUCATION

Hardin-Simmons University — Abilene, TX

Bachelor of Business Administration (BBA), Computer Science